
Práctica 7: Efectos de postprocesado

Objetivo: Postprocesado utilizando la Render Feature de URP

1. Instrucciones

[Igual que la práctica anterior]

La práctica se realiza sobre un proyecto de Unity en el que se van añadiendo escenas con distintos elementos y que utilizan shaders, materiales y renderers distintos.

Se entregará el proyecto completo de Unity, dejando únicamente los directorios necesarios (Assets, Packages y ProjectSettings). En el directorio Assets habrá, además, una carpeta Capturas con imágenes de pantalla que muestren el resultado de cada una de las tareas pedidas.

2. Punto de partida

Crea un proyecto 3D en Unity que utilice el *Universal Render Pipeline*. Añade un directorio URPSettings para sus assets. Antes de empezar con la primera parte:

- Crea un asset “*URP Asset (with Universal Renderer)*” en la carpeta anterior llamado PR07_URenderPipelineAsset. Creará también un *renderer* que se utilizará por defecto PR07_URenderPipelineAsset_Renderer.
- Configura el proyecto (opción *Graphics* y quizá en las *Quality Settings*) para que utilice esa configuración.

En la última parte de la práctica se utilizará ese renderer (será preferible *forzar su uso* en lugar de dejar el renderer por defecto, que cambiaremos para hacer pruebas cómodamente).

En la primera parte cada escena tendrá un renderer distinto, para lo que habrá que:

- Crear el nuevo renderer “*URP Universal Renderer*” con el nombre concreto indicado en la carpeta URPSettings.



Figura 1: Contenido del directorio Assets de la entrega

- Añadir el renderer en la lista de renderers de PR07_URenderPipelineAsset.
- Seleccionar ese renderer en la cámara de la escena.
- Aunque no es obligatorio, seguramente quieras poner el nuevo renderer como el renderer por defecto para que se utilice también en la pestaña *Scene* (si no has forzado el uso del renderer inicial en las primeras escenas, esta acción alterará su funcionamiento).

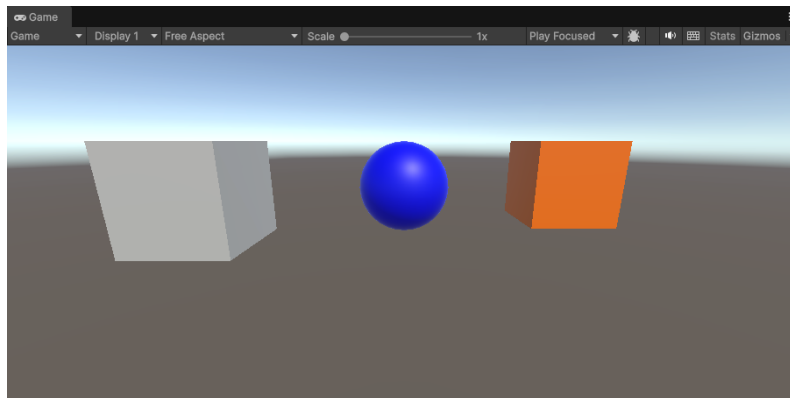
Crea en la carpeta Materials los siguientes materiales, basados en el shader por defecto de URP, Universal Render Pipeline/Lit:

- Azul que utilice el RGB (0, 0, 255).
- Naranja con (255, 120, 0).
- GrisClaro con (200, 200, 200).

Coloca en la escena los siguientes objetos:

- Cubo GrisClaro en L: (-2, 0.5, 0).
- Esfera Azul en L: (0, 0.5, 0.5).
- Cubo Naranja en L: (2, 0.5, 1).

Configura la cámara de la escena en L: (0, 1, -3) y R: (15, 0, 0).



3. Marca de agua

Tanto en este como en el resto de apartados haremos efectos de postprocesado que se añadirán a la escena gracias a la *Full Screen Pass Renderer Feature*.

El objetivo de esta práctica es añadir en la imagen final una “marca de agua” a partir de una textura con una imagen y un alpha.

Crea el shader, material, renderer y escena llamados 01MarcaAgua que lo hagan posible. Utiliza el fichero `MarcaDeAgua.png` proporcionado junto con el enunciado, que contiene el texto “ILM” en el centro de color rojo con borde azul y con un alpha que hace que la parte de atrás se vea ligeramente.

Implementa un shader que permita añadir una “marca de agua” en la imagen final.

4. Reflexión

Implementa un shader de postprocesado (02Reflection) que permita hacer un efecto de reflexión. Tendrá dos parámetros, ambos entre 0 y 1:

```
Properties
{
    _Horizontal("Horizontal flip", Range(0.0, 1.0)) = 0.0
    _Vertical("Vertical flip", Range(0.0, 1.0)) = 0.0
}
```

`_Horizontal` indica en qué posición horizontal debe hacerse la reflexión de forma que si, por ejemplo, vale 0.2, entonces el 20 % de la parte izquierda de la imagen contiene en realidad un reflejo del 20 % de la parte derecha. El parámetro `_Vertical` funciona de forma similar pero cambiando la parte inferior de la imagen.

A modo de ejemplo, la imagen 2 muestra una captura de la pestaña *Scene* del editor cuando el parámetro horizontal está en 0.2 y el vertical en 0.4.

5. Glitch

Implementa en 03Glitch un shader de postprocesado de *glitch* similar al de la práctica anterior pero que realice el efecto en la imagen final completa en lugar de asociarlo a objetos individuales.

Aunque lo habitual en estos efectos es que el desplazamiento sea en horizontal (como hicimos en una práctica anterior) modifícalo para que sea en vertical:

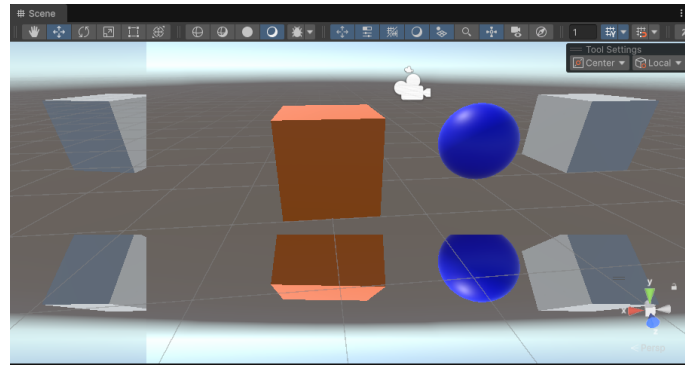


Figura 2: Pestaña *Scene* del editor con Reflection con 0.2 y 0.4

Properties

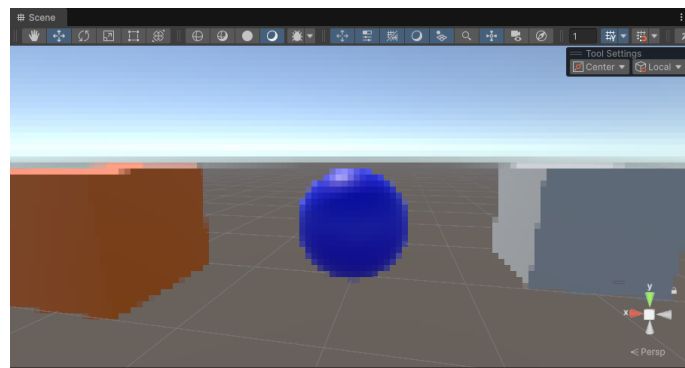
```
{
    [IntRange] _MaxDesplX("Maximo glitch vertical (pixeles)", Range(0, 100)) = 10
}
```

6. Pixelado

Crea un shader, material, renderer y escena nuevos, `04Pixelate`, para implementar el efecto de pixelar la imagen final, de forma que esté formada por bloques cuadrados de color uniforme extraído del color del pixel situado en el centro del bloque. El tamaño de cada bloque debe especificarse como parámetro del shader:

Properties

```
{
    [IntRange] _PixelSize("Tamaño de cada pixel", Range(0, 100)) = 1
}
```



7. Radial blur

En último efecto el color final de cada pixel necesitará leer de la imagen final 10 veces para conseguir un efecto de difuminación radial (`05RadialBlur`).

En los parámetros del efecto el centro de la difuminación y la escala:

Properties

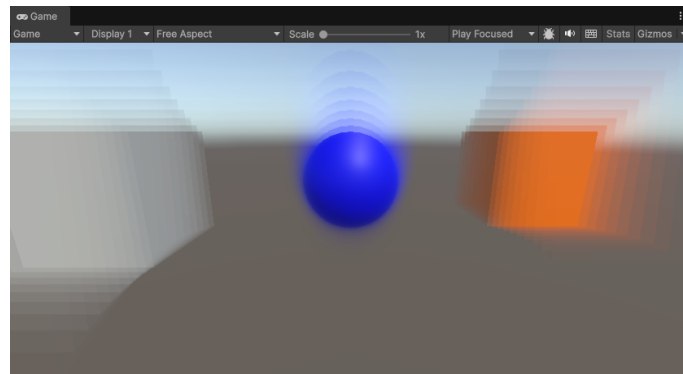
```
{
```

```
_CenterX("Center (X)", float) = 0.5  
_CenterY("Center (Y)", float) = 0.5  
_Scale("Scale", float) = 10  
}
```

Para cada pixel se leen 10 posiciones de la imagen generada de la siguiente forma:

- Se calcula la dirección en la que se tomarán las muestras restando el centro del efecto (parámetro) con el pixel.
- Se ajusta la longitud de la dirección de forma que coincida, en número de píxeles, con el parametro `_Scale`. Si la escala es 0, todas las muestras se cogerán en el mismo sitio, pues la dirección será 0.
- Se toman 10 muestras de la pantalla, empezando en la posición del pixel y desplazándose en la dirección calculada. El peso de cada muestra se irá reduciendo linealmente.

La captura siguiente muestra el efecto centrado en (0.5, 0.5) y con 30 de escala.



8. Entrega de la práctica

La práctica debe entregarse utilizando el mecanismo de entrega del campus virtual, no más tarde de la fecha y hora indicada en la cabecera de la práctica.

Solo uno de los dos miembros del grupo debe hacer la entrega, subiendo al campus un fichero llamado `GrupoNN.zip` donde NN representa el número de grupo con dos dígitos.

El fichero debe tener exclusivamente:

- El proyecto de Unity con los directorios mínimos necesarios (`Assets`, `Packages` y `ProjectSettings`). El contenido del directorio `Assets` debe ser el que se deduce de las instrucciones anteriores y que puede verse en la figura 1.
- Un fichero `alumnos.txt` donde se indicará el nombre de los componentes del grupo.